03-ACS-L-A3-S1: Calculatoare numerice II (Seria CA - 2021)

Colocviu Practic 1

Available from: Thursday, 27 January 2022, 8:35 AM
Due date: Thursday, 27 January 2022, 8:45 AM
Requested files: alu.v, bus_interface_unit.v, control_unit.v, cpu.v, decode_unit.v, defines.vh, gpio.v, gpio_unit.v, interrupt_controller.v, reg_file_interface_unit.v, register_file.v, rom.v, signal_generation_unit.v, sram.v, state_machine.v, timer_unit.v, cpu_test.v (Download)
Type of work: Individual work

Având scheletul de laborator implmentați următoarea instrucțiune: **SQUARE_SUM**

16-bit opcode: 0011_10rr_rrrd_dddd

Operation: Rd <- Rd^2 + Rr^2

Syntax: SQUARE_SUM Rd, Rr

Operands: 0 ≤ d ≤ 31, 0 ≤ r ≤ 31

Program counter: PC <- PC + 1

Flags:

Z -> R == 0
N -> R7

R (Result) equals Rd after the operation

Requested files

alu.v

```
`include "defines.vh"
module alu #(
        parameter DATA_WIDTH = 8
    )(
        input  wire [`OPSEL_COUNT-1:0] opsel,
        input  wire                    enable,
        input  wire   [DATA_WIDTH-1:0] rd,
        input  wire   [DATA_WIDTH-1:0] rr,
        output reg    [DATA_WIDTH-1:0] out,
        input  wire   [DATA_WIDTH-1:0] flags_in,
        output reg    [DATA_WIDTH-1:0] flags_out,
        input  wire                    cin_en,    // carry_in_enable
        input  wire                    cout_en    // carry_out_enable
    );

    // TODO 4: Implement the given instruction according to the description
    always @* begin
        if (enable)
            case (opsel)
            `OPSEL_ADD:
            begin
            if (cout_en) begin
               {flags_out[`FLAGS_C], out} = rd + rr + (cin_en & flags_in[`FLAGS_C]);
            end else begin
                                    out  = rd + rr + (cin_en & flags_in[`FLAGS_C]);
                flags_out[`FLAGS_C]       = flags_in[`FLAGS_C];
            end
                flags_out[`FLAGS_V] = (rd[7] == 1 && rr[7] == 1 && out[7] == 0) ||
                                      (rd[7] == 0 && rr[7] == 0 && out[7] == 1);
                flags_out[`FLAGS_Z] = (out == 0);
                flags_out[`FLAGS_N] = out[7];
                flags_out[`FLAGS_S] = flags_out[`FLAGS_N] ^ flags_out[`FLAGS_V];
                flags_out[`FLAGS_H] = (rd[3] == 1 && rr[3] == 1 && out[3] == 0) ||
                                      (rd[3] == 0 && rr[3] == 0 && out[3] == 1);
                flags_out[`FLAGS_T] = flags_in[`FLAGS_T];
                flags_out[`FLAGS_I] = flags_in[`FLAGS_I];
            end

            `OPSEL_SUB:
            begin
            if (cout_en) begin
               {flags_out[`FLAGS_C], out} = rd - rr - (cin_en & flags_in[`FLAGS_C]);
            end else begin
                                    out  = rd - rr - (cin_en & flags_in[`FLAGS_C]);
                flags_out[`FLAGS_C]       = flags_in[`FLAGS_C];
            end
                flags_out[`FLAGS_V] = (rd[7] == 1 && rr[7] == 1 && out[7] == 0) ||
                                      (rd[7] == 0 && rr[7] == 0 && out[7] == 1);
                flags_out[`FLAGS_Z] = (out == 0);
                flags_out[`FLAGS_N] = out[7];
                flags_out[`FLAGS_S] = flags_out[`FLAGS_N] ^ flags_out[`FLAGS_V];
                flags_out[`FLAGS_H] = (rd[3] == 1 && rr[3] == 1 && out[3] == 0) ||
                                      (rd[3] == 0 && rr[3] == 0 && out[3] == 1);
                flags_out[`FLAGS_T] = flags_in[`FLAGS_T];
                flags_out[`FLAGS_I] = flags_in[`FLAGS_I];
            end

            `OPSEL_AND:
            begin
                out = rd & rr;
                flags_out[`FLAGS_V] = 0;
                flags_out[`FLAGS_Z] = (out == 0);
                flags_out[`FLAGS_N] = out[7];
                flags_out[`FLAGS_S] = flags_out[`FLAGS_N] ^ flags_out[`FLAGS_V];
                flags_out[`FLAGS_H] = flags_in[`FLAGS_H];
                flags_out[`FLAGS_C] = flags_in[`FLAGS_C];
                flags_out[`FLAGS_T] = flags_in[`FLAGS_T];
                flags_out[`FLAGS_I] = flags_in[`FLAGS_I];
            end

            `OPSEL_XOR:
            begin
                out = rd ^ rr;
                flags_out[`FLAGS_V] = 0;
                flags_out[`FLAGS_Z] = (out == 0);
                flags_out[`FLAGS_N] = out[7];
                flags_out[`FLAGS_S] = flags_out[`FLAGS_N] ^ flags_out[`FLAGS_V];
                flags_out[`FLAGS_H] = flags_in[`FLAGS_H];
                flags_out[`FLAGS_C] = flags_in[`FLAGS_C];
                flags_out[`FLAGS_T] = flags_in[`FLAGS_T];
                flags_out[`FLAGS_I] = flags_in[`FLAGS_I];
            end

            `OPSEL_OR:
            begin
                out = rd | rr;
                flags_out[`FLAGS_V] = 0;
                flags_out[`FLAGS_Z] = (out == 0);
                flags_out[`FLAGS_N] = out[7];
                flags_out[`FLAGS_S] = flags_out[`FLAGS_N] ^ flags_out[`FLAGS_V];
                flags_out[`FLAGS_H] = flags_in[`FLAGS_H];
                flags_out[`FLAGS_C] = flags_in[`FLAGS_C];
                flags_out[`FLAGS_T] = flags_in[`FLAGS_T];
                flags_out[`FLAGS_I] = flags_in[`FLAGS_I];
            end

            `OPSEL_NEG:
            begin
                out = ~rd;
                flags_out[`FLAGS_V] = 0;
                flags_out[`FLAGS_Z] = (out == 0);
                flags_out[`FLAGS_N] = out[7];
                flags_out[`FLAGS_S] = flags_out[`FLAGS_N] ^ flags_out[`FLAGS_V];
```

```
104                 flags_out[`FLAGS_H] = flags_in[`FLAGS_H];
105                 flags_out[`FLAGS_C] = flags_in[`FLAGS_C];
106                 flags_out[`FLAGS_T] = flags_in[`FLAGS_T];
107                 flags_out[`FLAGS_I] = flags_in[`FLAGS_I];
108             end
109 
110             `OPSEL_NONE: begin
111                 out = rr;
112                 flags_out = flags_in;
113             end
114 
115             default:
116             begin
117                 out = {DATA_WIDTH{1'bx}};
118                 flags_out = flags_in;
119             end
120             endcase
121         else /* if alu_enable == false */ begin
122             out = {DATA_WIDTH{1'bx}};
123             flags_out = flags_in;
124         end
125     end
126 
127 endmodule
128 
```

bus_interface_unit.v

```
`include "defines.vh"
module bus_interface_unit #(
        parameter MEM_START_ADDR = 8'h40,
        parameter MEM_STOP_ADDR  = 8'hBF,
        parameter IO_START_ADDR  = 8'h00,
        parameter IO_STOP_ADDR   = 8'h3F,
        parameter DATA_WIDTH     = 8,    // registers are 8 bits in width
        parameter ADDR_WIDTH     = 16    // 64KB address space
    )(
        input  wire  [`GROUP_COUNT-1:0] opcode_group,
        input  wire             [11:0] opcode_imd,
        input  wire [`SIGNAL_COUNT-1:0] signals,
        input  wire                     cycle_count,
        input  wire    [DATA_WIDTH-1:0] data_to_store,
        input  wire    [ADDR_WIDTH-1:0] indirect_addr,
        output wire    [ADDR_WIDTH-1:0] bus_addr,
        inout  wire    [DATA_WIDTH-1:0] bus_data,
        output wire                     mem_cs,
        output wire                     mem_we,
        output wire                     mem_oe,
        output wire                     io_cs,
        output wire                     io_we,
        output wire                     io_oe
    );

    wire [ADDR_WIDTH-1:0] internal_mem_addr;
    wire [ADDR_WIDTH-1:0] internal_io_addr;
    wire                  mem_access, io_access;
    wire                  mem_addr_is_in_mem;
    wire                  mem_addr_is_in_io;
    wire                  uses_indirect;
    wire                  should_store;
    wire                  should_load;

    assign should_load =
        signals[`CONTROL_MEM_READ] || signals[`CONTROL_IO_READ];
    assign should_store =
        signals[`CONTROL_MEM_WRITE] || signals[`CONTROL_IO_WRITE];
    assign uses_indirect =
        (opcode_group[`GROUP_LOAD_INDIRECT] ||
         opcode_group[`GROUP_STORE_INDIRECT]);
    assign mem_access = signals[`CONTROL_MEM_READ] ||
                        signals[`CONTROL_MEM_WRITE];
    assign io_access  = signals[`CONTROL_IO_READ]  ||
                        signals[`CONTROL_IO_WRITE];
    assign mem_addr_is_in_mem = mem_access &&
            (internal_mem_addr >= MEM_START_ADDR &&
             internal_mem_addr <= MEM_STOP_ADDR);
    assign mem_addr_is_in_io  = mem_access &&
            (internal_mem_addr >= IO_START_ADDR &&
             internal_mem_addr <= IO_STOP_ADDR);

    /* Stack operations: push, pop, rcall, ret should access SPL.
     * But call_isr and reti, which are 2-cycle write instructions,
     * should also access SREG on their second cycle */
    assign internal_io_addr =
        io_access ?
            opcode_group[`GROUP_STACK] ? (cycle_count == 0) ? `SPL : `SREG :
            opcode_group[`GROUP_ALU]   ? `SREG :
            {4'b0, opcode_imd} :
        mem_addr_is_in_io ?
            internal_mem_addr :
        {ADDR_WIDTH{1'bx}};
    assign internal_mem_addr =
        uses_indirect ?
            indirect_addr :
            {4'b0, opcode_imd};

    assign mem_cs = mem_addr_is_in_mem;
    assign mem_we = (mem_cs && signals[`CONTROL_MEM_WRITE]) ? 1'b1 :
                    (mem_cs && signals[`CONTROL_MEM_READ])  ? 1'b0 : 1'bx;
    assign mem_oe = (mem_cs && signals[`CONTROL_MEM_READ])  ? 1'b1 : 1'bx;

    assign io_cs = io_access || mem_addr_is_in_io;
    assign io_we = (io_cs && should_store) ? 1'b1 :
                   (io_cs && should_load)  ? 1'b0 : 1'bx;
    assign io_oe = (io_cs && should_load)  ? 1'b1 : 1'bx;

    assign bus_addr = mem_cs ? internal_mem_addr - MEM_START_ADDR:
                      io_cs  ? internal_io_addr - IO_START_ADDR :
                      {ADDR_WIDTH{1'bx}};
    assign bus_data = should_store ? data_to_store : {DATA_WIDTH{1'bz}};

endmodule

```

control_unit.v

```
`include "defines.vh"
module control_unit #(
        parameter  INSTR_WIDTH = 16,   // instructions are 16 bits in width
        parameter   DATA_WIDTH = 8,    // registers are 8 bits in width
        parameter I_ADDR_WIDTH = 10,   // 2*1024 bytes of flash (or ROM in our case)
        parameter   ADDR_WIDTH = 16,   // 64KB address space
        parameter D_ADDR_WIDTH = 7,    // 128 bytes of SRAM
        parameter R_ADDR_WIDTH = 5     // 32 registers
    )(
        input  wire                    clk,
        input  wire                    reset,
        // To/from instruction memory
        output reg  [I_ADDR_WIDTH-1:0] program_counter,
        input  wire  [INSTR_WIDTH-1:0] instruction,
        // From FSM
        output wire [`STATE_COUNT-1:0] state,
        // To/from interrupt controller
        input  wire                    irq,
        input  wire [I_ADDR_WIDTH-1:0] vector,
        output reg                     ack,
        // To/from register file
        output wire [R_ADDR_WIDTH-1:0] rr_addr,
        output wire [R_ADDR_WIDTH-1:0] rd_addr,
        inout  wire   [DATA_WIDTH-1:0] rr_data,
        inout  wire   [DATA_WIDTH-1:0] rd_data,
        output wire                    rr_cs,
        output wire                    rd_cs,
        output wire                    rr_we,
        output wire                    rd_we,
        output wire                    rr_oe,
        output wire                    rd_oe,
        // To/from ALU
        output wire                    alu_enable,
        output wire                    alu_cin_en,
        output wire                    alu_cout_en,
        output reg  [`OPSEL_COUNT-1:0] alu_opsel,
        output wire   [DATA_WIDTH-1:0] alu_flags_in,
        input  wire   [DATA_WIDTH-1:0] alu_flags_out,
        output reg    [DATA_WIDTH-1:0] alu_rr,
        output reg    [DATA_WIDTH-1:0] alu_rd,
        input  wire   [DATA_WIDTH-1:0] alu_out,
        // To/from bus interface unit
        output wire   [ADDR_WIDTH-1:0] bus_addr,
        inout  wire   [DATA_WIDTH-1:0] bus_data,
        output wire                    mem_cs,
        output wire                    mem_we,
        output wire                    mem_oe,
        output wire                    io_cs,
        output wire                    io_we,
        output wire                    io_oe
    );

    // From decode unit and signal generation unit
    wire [`SIGNAL_COUNT-1:0] signals;
    wire [`OPCODE_COUNT-1:0] opcode_type;
    wire  [`GROUP_COUNT-1:0] opcode_group;
    wire  [R_ADDR_WIDTH-1:0] opcode_rd;
    wire  [R_ADDR_WIDTH-1:0] opcode_rr;
    wire              [11:0] opcode_imd;
    wire               [2:0] opcode_bit;
    wire                     cycle_count;
    // Buffers for various stuff
    reg    [INSTR_WIDTH-1:0] instr_buffer;
    reg     [DATA_WIDTH-1:0] alu_out_buffer;
    reg     [DATA_WIDTH-1:0] writeback_value;
    wire    [ADDR_WIDTH-1:0] indirect_addr;
    wire    [DATA_WIDTH-1:0] data_to_store;
    reg     [DATA_WIDTH-1:0] sreg;
    reg     [DATA_WIDTH-1:0] sp;
    reg     [ADDR_WIDTH-1:0] saved_pc;
    reg                      irq_buffer;
    wire                     alu_save_flags;

    state_machine fsm (
        .state       (state),
        .cycle_count (cycle_count),
        .opcode_group(opcode_group),
        .clk         (clk),
        .reset       (reset)
    );

    decode_unit #(
        .INSTR_WIDTH(INSTR_WIDTH)
    ) decode (
        .instruction (instr_buffer),
        .opcode_type (opcode_type),
        .opcode_group(opcode_group),
        .opcode_imd  (opcode_imd),
        .opcode_rd   (opcode_rd),
        .opcode_rr   (opcode_rr),
        .opcode_bit  (opcode_bit),
        .irq         (irq_buffer)
    );

    signal_generation_unit sig (
        .state       (state),
        .cycle_count (cycle_count),
        .opcode_type (opcode_type),
        .opcode_group(opcode_group),
        .signals     (signals)
    );

    reg_file_interface_unit #(
```

```
        .DATA_WIDTH  (DATA_WIDTH),
        .INSTR_WIDTH (INSTR_WIDTH),
        .D_ADDR_WIDTH(D_ADDR_WIDTH),
        .R_ADDR_WIDTH(R_ADDR_WIDTH)
    ) rf_int (
        .opcode_type    (opcode_type),
        .opcode_rd      (opcode_rd),
        .opcode_rr      (opcode_rr),
        .signals        (signals),
        .cycle_count    (cycle_count),
        .writeback_value(writeback_value),
        .rr_addr        (rr_addr),
        .rd_addr        (rd_addr),
        .rr_data        (rr_data),
        .rd_data        (rd_data),
        .rr_cs          (rr_cs),
        .rd_cs          (rd_cs),
        .rr_we          (rr_we),
        .rd_we          (rd_we),
        .rr_oe          (rr_oe),
        .rd_oe          (rd_oe)
    );

    bus_interface_unit #(
        .MEM_START_ADDR(8'h40),
        .MEM_STOP_ADDR (8'hBF),
        .IO_START_ADDR (8'h00),
        .IO_STOP_ADDR  (8'h3F),
        .DATA_WIDTH    (DATA_WIDTH),
        .ADDR_WIDTH    (ADDR_WIDTH)
    ) bus_int (
        .opcode_group (opcode_group),
        .opcode_imd   (opcode_imd),
        .signals      (signals),
        .cycle_count  (cycle_count),
        .bus_addr     (bus_addr),
        .bus_data     (bus_data),
        .mem_cs       (mem_cs),
        .mem_we       (mem_we),
        .mem_oe       (mem_oe),
        .io_cs        (io_cs),
        .io_we        (io_we),
        .io_oe        (io_oe),
        .indirect_addr(indirect_addr),
        .data_to_store(data_to_store)
    );

    assign indirect_addr =
        (opcode_group[`GROUP_LOAD_INDIRECT] ||
         opcode_group[`GROUP_STORE_INDIRECT]) ?
            (opcode_group[`GROUP_STACK] ?
                (cycle_count == 0) ? {8'b0, sp} : `SREG :
        // else, indirect to memory => X or Y or Z
                {alu_rr, alu_rd}) :
        // else, not indirect
            {ADDR_WIDTH{1'bx}};

    assign data_to_store =
            signals[`CONTROL_MEM_WRITE] ?
                opcode_type == `TYPE_RCALL || opcode_type == `TYPE_CALL_ISR ?
                    cycle_count ? saved_pc[7:0]  :
                                  saved_pc[15:8] :
                alu_rr :
            signals[`CONTROL_IO_WRITE] ?
                opcode_group[`GROUP_STACK]   ? (cycle_count == 0) ? sp : sreg :
                opcode_group[`GROUP_ALU]     ? sreg :
                opcode_group[`GROUP_ALU_AUX] ? alu_out_buffer : // write modified value back to I/O space
                opcode_type == `TYPE_OUT     ? alu_rr :
                {DATA_WIDTH{1'bx}} :
            opcode_group[`GROUP_ALU_AUX] ?
                alu_out_buffer :
            {DATA_WIDTH{1'bx}};

    assign alu_enable = (state == `STATE_EX) &&
                        (opcode_group[`GROUP_ALU]     ||
                         opcode_group[`GROUP_ALU_IMD] ||
                         opcode_group[`GROUP_ALU_AUX]);

    assign alu_flags_in = sreg;
    /* Bloc de atribuire al sreg-ului */
    always @(posedge clk, posedge reset)
        if (reset)
            sreg <= 0;
        else if (bus_addr == `SREG)
            sreg <= bus_data;
        else if (alu_enable && alu_save_flags)
            sreg <= alu_flags_out;
        else if (state == `STATE_EX && (opcode_type == `TYPE_CALL_ISR || opcode_type == `TYPE_RETI))
            sreg <= alu_out;

    /* Bloc de atribuire al sp-ului */
    always @(posedge clk, posedge reset) begin
        if (reset)
            sp <= 0;
        else if (bus_addr == `SPL)
            sp <= bus_data;
        else
            if (signals[`CONTROL_POSTDEC])
                sp <= sp - 8'b1;
            else if (signals[`CONTROL_PREINC])
                sp <= sp + 8'b1;
    end

    /* Bloc de atribuire al program counter-ului */
```

```
208     always @(posedge clk, posedge reset) begin
209         if (reset) begin
210             program_counter <= 0;
211         end else if (state == `STATE_EX) begin
212             case (opcode_type)
213             `TYPE_RJMP: begin
214                 program_counter <= program_counter + opcode_imd[9:0];
215             end
216             `TYPE_BRBS: begin
217                 if (sreg[opcode_bit] == 1)
218                     program_counter <= program_counter + opcode_imd[9:0];
219             end
220             `TYPE_BRBC: begin
221                 if (sreg[opcode_bit] == 0)
222                     program_counter <= program_counter + opcode_imd[9:0];
223             end
224             `TYPE_RCALL: begin
225                 program_counter <= program_counter + opcode_imd[9:0];
226             end
227             `TYPE_CALL_ISR: begin
228                 program_counter <= vector - 2;
229             end
230             endcase
231         end else if (state == `STATE_WB) begin
232             if (opcode_type == `TYPE_RET || opcode_type == `TYPE_RETI) begin
233                 program_counter <= saved_pc[9:0];
234             end else begin
235                 program_counter <= program_counter + 10'b1;
236             end
237         end
238     end
239
240     always @(posedge clk, posedge reset) begin
241         if (reset)
242             saved_pc <= 0;
243         else if (state == `STATE_ID)
244         begin
245             if (opcode_type == `TYPE_RCALL) begin
246                 saved_pc <= {6'b0, program_counter} + 12'b1;
247             end else if (opcode_type == `TYPE_CALL_ISR) begin
248                 saved_pc <= {6'b0, program_counter};
249             end
250         end
251         else if (state == `STATE_MEM &&
252                 (opcode_type == `TYPE_RET || opcode_type == `TYPE_RETI))
253         begin
254             if (cycle_count == 0) begin
255                 saved_pc[7:0]  <= bus_data;
256             end else begin
257                 saved_pc[15:8] <= bus_data;
258             end
259         end
260     end
261
262     always @(posedge clk, posedge reset) begin
263         if (reset)
264             writeback_value <= {DATA_WIDTH{1'b0}};
265         else if (opcode_type == `TYPE_LDI)
266             writeback_value <= opcode_imd[DATA_WIDTH-1:0];
267         else if (signals[`CONTROL_IO_READ] ||
268                  signals[`CONTROL_MEM_READ])
269             writeback_value <= bus_data;
270         else if (opcode_group[`GROUP_ALU] || opcode_group[`GROUP_ALU_IMD])
271             writeback_value <= alu_out_buffer;
272         else if (opcode_type == `TYPE_MOV)
273             writeback_value <= alu_rr;
274     end
275
276     /* Buffer pentru instructiunea citita */
277     always @(posedge clk, posedge reset)
278         if (reset) begin
279             instr_buffer <= 0;
280         end else if (state == `STATE_IF) begin
281             instr_buffer <= instruction;
282         end
283
284     /* Buffer pentru output-ul UAL-ului */
285     always @(posedge clk, posedge reset)
286         if (reset) begin
287             alu_out_buffer <= 0;
288         end else if (state == `STATE_EX) begin
289             alu_out_buffer <= alu_out;
290         end
291
292     always @(posedge clk, posedge reset)
293         if (reset) begin
294             alu_rd <= 0;
295             alu_rr <= 0;
296         end else if (state == `STATE_ID) begin
297             if (opcode_type == `TYPE_SBI || opcode_type == `TYPE_SEI)
298             begin
299                 alu_rd <= bus_data;
300                 alu_rr <= (8'b1 << opcode_bit);
301             end else if (opcode_type == `TYPE_CBI || opcode_type == `TYPE_CLI)
302             begin
303                 alu_rd <= bus_data;
304                 alu_rr <= ~(8'b1 << opcode_bit);
305             end else if (opcode_type == `TYPE_INC ||
306                          opcode_type == `TYPE_DEC)
307             begin
308                 alu_rd <= rd_data;
309                 alu_rr <= 1;
310             end else if (opcode_group[`GROUP_ALU_IMD])
```

```
311             begin
312                 alu_rd <= rd_data;
313                 alu_rr <= opcode_imd;
314             end else if (opcode_type == `TYPE_CALL_ISR)
315             begin
316                 alu_rd <= sreg;
317                 alu_rr <= ~(8'b1 << `FLAGS_I);
318             end else if (opcode_type == `TYPE_RETI)
319             begin
320                 alu_rd <= sreg;
321                 alu_rr <= (8'b1 << `FLAGS_I);
322             end else begin
323                 alu_rd <= rd_data;
324                 alu_rr <= rr_data;
325             end
326         end
327
328     /* TODO3: Set alu_opsel to appropriate operation,
329      * according to opcode_type and alu_enable for the given instruction*/
330     always @* begin
331         if (alu_enable == 0)
332             alu_opsel = `OPSEL_COUNT'bx;
333         else begin
334             case (opcode_type)
335             `TYPE_ADD, `TYPE_ADC, `TYPE_INC:
336                 alu_opsel = `OPSEL_ADD;
337             `TYPE_SUB, `TYPE_DEC:
338                 alu_opsel = `OPSEL_SUB;
339             `TYPE_AND, `TYPE_CBI, `TYPE_CLI, `TYPE_CALL_ISR:
340                 alu_opsel = `OPSEL_AND;
341             `TYPE_EOR:
342                 alu_opsel = `OPSEL_XOR;
343             `TYPE_OR, `TYPE_SBI, `TYPE_SEI, `TYPE_RETI:
344                 alu_opsel = `OPSEL_OR;
345             `TYPE_NEG:
346                 alu_opsel = `OPSEL_NEG;
347             default:
348                 alu_opsel = `OPSEL_NONE;
349             endcase
350         end
351     end
352
353     assign alu_cin_en  = alu_enable && (opcode_type == `TYPE_ADC);
354     assign alu_cout_en = alu_enable && (opcode_type != `TYPE_INC) && (opcode_type != `TYPE_DEC);
355     assign alu_save_flags = (opcode_group[`GROUP_ALU_AUX] == 0);
356
357     always @(posedge clk, posedge reset) begin
358         if (reset) begin
359             irq_buffer <= 0;
360         end else if (state == `STATE_IF) begin
361             irq_buffer <= irq;
362         end
363     end
364
365     always @(posedge clk, posedge reset) begin
366         if (reset) begin
367             ack <= 0;
368         end else if (state == `STATE_WB) begin
369             ack <= opcode_type == `TYPE_CALL_ISR;
370         end else begin
371             ack <= 0;
372         end
373     end
374 endmodule
375
```

cpu.v

```
`include "defines.vh"
module cpu #(
        parameter      INSTR_WIDTH  = 16,   // instructions are 16 bits in width
        parameter       DATA_WIDTH  = 8,    // registers are 8 bits in width
        parameter     I_ADDR_WIDTH  = 10,   // 2 * 1024 bytes of flash (or ROM in our case)
        parameter       ADDR_WIDTH  = 16,   // 64KB address space
        parameter     D_ADDR_WIDTH  = 7,    // 128 bytes of SRAM
        parameter    IO_ADDR_WIDTH  = 6,    // 64 I/O registers
        parameter     R_ADDR_WIDTH  = 5,    // 32 registers
        parameter RST_ACTIVE_LEVEL  = 1     // level on which reset is active
    )(
        input  wire                    osc_clk,
        input  wire                    trace_clk,
        input  wire                    trace_mode,
        input  wire              [4:0] prescaler,
        input  wire                    reset,
        inout  wire   [DATA_WIDTH-1:0] pa,
        inout  wire   [DATA_WIDTH-1:0] pb,
        output wire                    oc0a,
        output wire                    oc0b,
        output wire [I_ADDR_WIDTH-1:0] debug_pc,
        output wire [`STATE_COUNT-1:0] debug_state
    );

    wire [`STATE_COUNT-1:0] state;
    wire [I_ADDR_WIDTH-1:0] program_counter;
    wire  [INSTR_WIDTH-1:0] instruction;
    wire [R_ADDR_WIDTH-1:0] rr_addr;
    wire [R_ADDR_WIDTH-1:0] rd_addr;
    wire   [DATA_WIDTH-1:0] rr_data;
    wire   [DATA_WIDTH-1:0] rd_data;
    wire                    rr_cs;
    wire                    rd_cs;
    wire                    rr_we;
    wire                    rd_we;
    wire                    rr_oe;
    wire                    rd_oe;
    wire                    alu_cin_en;
    wire                    alu_cout_en;
    wire                    alu_enable;
    wire [`OPSEL_COUNT-1:0] alu_opsel;
    wire   [DATA_WIDTH-1:0] alu_rr;
    wire   [DATA_WIDTH-1:0] alu_rd;
    wire   [DATA_WIDTH-1:0] alu_out;
    wire   [DATA_WIDTH-1:0] alu_flags_in;
    wire   [DATA_WIDTH-1:0] alu_flags_out;
    wire   [ADDR_WIDTH-1:0] bus_addr;
    wire   [DATA_WIDTH-1:0] bus_data;
    wire                    mem_cs;
    wire                    mem_we;
    wire                    mem_oe;
    wire                    io_cs;
    wire                    io_we;
    wire                    io_oe;
    reg              [31:0] clk_counter;
    wire                    clk_ps;
    wire                    clk;
    wire                    corrected_reset;
    wire                    irq;
    wire [I_ADDR_WIDTH-1:0] vector;
    wire                    ack;


    always @(posedge osc_clk, posedge corrected_reset)
        if (corrected_reset)
            clk_counter <= 0;
        else
            clk_counter <= clk_counter + 1;

    assign corrected_reset = (reset == RST_ACTIVE_LEVEL);

    assign clk_ps =  (prescaler == 0) ? osc_clk : clk_counter[prescaler - 1];
    assign clk    = (trace_mode == 0) ? clk_ps  : trace_clk;

    assign debug_pc = program_counter;
    assign debug_state = state;

    control_unit #(
        .DATA_WIDTH  (DATA_WIDTH),
        .ADDR_WIDTH  (ADDR_WIDTH),
        .D_ADDR_WIDTH(D_ADDR_WIDTH),
        .I_ADDR_WIDTH(I_ADDR_WIDTH),
        .R_ADDR_WIDTH(R_ADDR_WIDTH),
        .INSTR_WIDTH (INSTR_WIDTH)
    ) control (
        .program_counter(program_counter),
        .instruction    (instruction),
        .state          (state),
        .irq            (irq),
        .vector         (vector),
        .ack            (ack),
        .clk            (clk),
        .reset          (corrected_reset),
        .rr_addr        (rr_addr),
        .rd_addr        (rd_addr),
        .rr_data        (rr_data),
        .rd_data        (rd_data),
        .rr_cs          (rr_cs),
        .rd_cs          (rd_cs),
        .rr_we          (rr_we),
        .rd_we          (rd_we),
        .rr_oe          (rr_oe),
        .rd_oe          (rd_oe),
```

```
104         .alu_cin_en     (alu_cin_en),
105         .alu_cout_en    (alu_cout_en),
106         .alu_enable     (alu_enable),
107         .alu_opsel      (alu_opsel),
108         .alu_flags_in   (alu_flags_in),
109         .alu_flags_out  (alu_flags_out),
110         .alu_rr         (alu_rr),
111         .alu_rd         (alu_rd),
112         .alu_out        (alu_out),
113         .bus_addr       (bus_addr),
114         .bus_data       (bus_data),
115         .mem_cs         (mem_cs),
116         .mem_we         (mem_we),
117         .mem_oe         (mem_oe),
118         .io_cs          (io_cs),
119         .io_we          (io_we),
120         .io_oe          (io_oe)
121     );
122
123     alu #(
124         .DATA_WIDTH(DATA_WIDTH)
125     ) ual (
126         .opsel    (alu_opsel),
127         .enable   (alu_enable),
128         .rd       (alu_rd),
129         .rr       (alu_rr),
130         .out      (alu_out),
131         .flags_in (alu_flags_in),
132         .flags_out(alu_flags_out),
133         .cin_en   (alu_cin_en),
134         .cout_en  (alu_cout_en)
135     );
136
137     dual_port_sram #(
138         .DATA_WIDTH(DATA_WIDTH),
139         .ADDR_WIDTH(R_ADDR_WIDTH)
140     ) reg_file (
141         .clk    (clk),
142         .rr_addr(rr_addr),
143         .rd_addr(rd_addr),
144         .rr_data(rr_data),
145         .rd_data(rd_data),
146         .rr_cs  (rr_cs),
147         .rd_cs  (rd_cs),
148         .rr_we  (rr_we),
149         .rd_we  (rd_we),
150         .rr_oe  (rr_oe),
151         .rd_oe  (rd_oe)
152     );
153
154     rom #(
155         .DATA_WIDTH(INSTR_WIDTH),
156         .ADDR_WIDTH(I_ADDR_WIDTH)
157     ) instruction_mem (
158         .clk (clk),
159         .addr(program_counter),
160         .data(instruction)
161     );
162
163     sram #(
164         .ADDR_WIDTH(D_ADDR_WIDTH),
165         .DATA_WIDTH(DATA_WIDTH)
166     ) data_mem (
167         .clk    (clk),
168         .oe     (mem_oe),
169         .cs     (mem_cs),
170         .we     (mem_we),
171         .address(bus_addr[D_ADDR_WIDTH-1:0]),
172         .data   (bus_data)
173     );
174
175     io_sram #(
176         .ADDR_WIDTH(IO_ADDR_WIDTH),
177         .DATA_WIDTH(DATA_WIDTH)
178     ) io (
179         .clk    (clk),
180         .reset  (reset),
181         .oe     (io_oe),
182         .cs     (io_cs),
183         .we     (io_we),
184         .address(bus_addr[IO_ADDR_WIDTH-1:0]),
185         .data   (bus_data),
186         .pa     (pa),
187         .pb     (pb),
188         .oc0a   (oc0a),
189         .oc0b   (oc0b),
190         .ack    (ack),
191         .irq    (irq),
192         .vector (vector)
193     );
194
195 endmodule
196
```

decode_unit.v

```
`include "defines.vh"
module decode_unit #(
        parameter  INSTR_WIDTH  = 16,
        parameter  R_ADDR_WIDTH = 5
    )(
        input  wire   [INSTR_WIDTH-1:0] instruction,
        output reg  [`OPCODE_COUNT-1:0] opcode_type,
        output wire  [`GROUP_COUNT-1:0] opcode_group,
        output reg   [R_ADDR_WIDTH-1:0] opcode_rd,
        output reg   [R_ADDR_WIDTH-1:0] opcode_rr,
        output reg               [11:0] opcode_imd,
        output reg                [2:0] opcode_bit,
        input  wire                     irq
    );

    always @* begin
        if (irq) begin
            opcode_type = `TYPE_CALL_ISR;
            opcode_rd   = {R_ADDR_WIDTH{1'bx}};
            opcode_rr   = {R_ADDR_WIDTH{1'bx}};
            opcode_bit  = 3'bx;
            opcode_imd  = 12'bx;
        end else begin
            casez (instruction)
                16'b0000_11??_????_????: begin
                    opcode_type = `TYPE_ADD;
                    opcode_rd   = instruction[8:4];
                    opcode_rr   = {instruction[9], instruction[3:0]};
                    opcode_bit  = 3'bx;
                    opcode_imd  = 12'bx;
                end
                16'b0001_11??_????_????: begin
                    opcode_type = `TYPE_ADC;
                    opcode_rd   = instruction[8:4];
                    opcode_rr   = {instruction[9], instruction[3:0]};
                    opcode_bit  = 3'bx;
                    opcode_imd  = 12'bx;
                end
                16'b0010_00??_????_????: begin
                    opcode_type = `TYPE_AND;
                    opcode_rd   = instruction[8:4];
                    opcode_rr   = {instruction[9], instruction[3:0]};
                    opcode_bit  = 3'bx;
                    opcode_imd  = 12'bx;
                end

                // TODO 1: decode the given instruction
                /*
                16'b????_????_????_????: begin
                    opcode_type = ?;
                    opcode_rd   = ?;
                    opcode_rr   = ?;
                    opcode_bit  = ?;
                    opcode_imd  = ?;
                end
                */

                16'b1111_01??_????_????: begin
                    opcode_type = `TYPE_BRBC;
                    opcode_rd   = {R_ADDR_WIDTH{1'bx}};
                    opcode_rr   = {R_ADDR_WIDTH{1'bx}};
                    opcode_bit  = instruction[2:0];
                    opcode_imd  = {{5{instruction[9]}}, instruction[9:3]};
                end
                16'b1111_00??_????_????: begin
                    opcode_type = `TYPE_BRBS;
                    opcode_rd   = {R_ADDR_WIDTH{1'bx}};
                    opcode_rr   = {R_ADDR_WIDTH{1'bx}};
                    opcode_bit  = instruction[2:0];
                    opcode_imd  = {{5{instruction[9]}}, instruction[9:3]};
                end
                16'b1001_1000_????_????: begin
                    opcode_type = `TYPE_CBI;
                    opcode_rd   = {R_ADDR_WIDTH{1'bx}};
                    opcode_rr   = {R_ADDR_WIDTH{1'bx}};
                    opcode_bit  = instruction[2:0];
                    opcode_imd  = {7'b0, instruction[7:3]};
                end
                16'b1001_0100_1111_1000: begin
                    opcode_type = `TYPE_CLI;
                    opcode_rd   = {R_ADDR_WIDTH{1'bx}};
                    opcode_rr   = {R_ADDR_WIDTH{1'bx}};
                    opcode_bit  = `FLAGS_I;
                    opcode_imd  = `SREG;
                end
                16'b0001_01??_????_????: begin
                    opcode_type = `TYPE_CP;
                    opcode_rd   = instruction[8:4];
                    opcode_rr   = {instruction[9], instruction[3:0]};
                    opcode_bit  = 3'bx;
                    opcode_imd  = 12'bx;
                end
                16'b1001_010?_????_1010: begin
                    opcode_type = `TYPE_DEC;
                    opcode_rd   = instruction[8:4];
                    opcode_rr   = {R_ADDR_WIDTH{1'bx}};
                    opcode_bit  = 3'bx;
                    opcode_imd  = 12'bx;
                end
                16'b0010_01??_????_????: begin
                    opcode_type = `TYPE_EOR;
                    opcode_rd   = instruction[8:4];
                    opcode_rr   = {instruction[9], instruction[3:0]};
```

```
104                     opcode_bit  = 3'bx;
105                     opcode_imd  = 12'bx;
106                 end
107                 16'b1011_0???_????_????: begin
108                     opcode_type = `TYPE_IN;
109                     opcode_rd   = instruction[8:4];
110                     opcode_rr   = {R_ADDR_WIDTH{1'bx}};
111                     opcode_bit  = 3'bx;
112                     opcode_imd  = {instruction[10:9], instruction[3:0]};
113                 end
114                 16'b1001_010?_????_0011: begin
115                     opcode_type = `TYPE_INC;
116                     opcode_rd   = instruction[8:4];
117                     opcode_rr   = {R_ADDR_WIDTH{1'bx}};
118                     opcode_bit  = 3'bx;
119                     opcode_imd  = 12'bx;
120                 end
121                 16'b1000_000?_????_1000: begin
122                     opcode_type = `TYPE_LD_Y;
123                     opcode_rd   = instruction[8:4];
124                     opcode_rr   = {R_ADDR_WIDTH{1'bx}};
125                     opcode_bit  = 3'bx;
126                     opcode_imd  = 12'bx;
127                 end
128                 16'b1110_????_????_????: begin
129                     opcode_type = `TYPE_LDI;
130                     opcode_rd   = {1'b1, instruction[7:4]};
131                     opcode_rr   = {R_ADDR_WIDTH{1'bx}};
132                     opcode_bit  = 3'bx;
133                     opcode_imd  = {4'b0, instruction[11:8], instruction[3:0]};
134                 end
135                 16'b1010_0???_????_????: begin
136                     opcode_type = `TYPE_LDS;
137                     opcode_rd   = {1'b1, instruction[7:4]};
138                     opcode_rr   = {R_ADDR_WIDTH{1'bx}};
139                     opcode_bit  = 3'bx;
140                     opcode_imd  = {~instruction[8], instruction[8],
141                         instruction[10:9], instruction[3:0]};
142                 end
143                 16'b0010_11??_????_????: begin
144                     opcode_type = `TYPE_MOV;
145                     opcode_rd   = instruction[8:4];
146                     opcode_rr   = {instruction[9], instruction[3:0]};
147                     opcode_bit  = 3'bx;
148                     opcode_imd  = 12'bx;
149                 end
150                 16'b1001_010?_????_0001: begin
151                     opcode_type = `TYPE_NEG;
152                     opcode_rd   = instruction[8:4];
153                     opcode_rr   = {R_ADDR_WIDTH{1'bx}};
154                     opcode_bit  = 3'bx;
155                     opcode_imd  = 12'bx;
156                 end
157                 16'b0000_0000_0000_0000: begin
158                     opcode_type = `TYPE_NOP;
159                     opcode_rd   = {R_ADDR_WIDTH{1'bx}};
160                     opcode_rr   = {R_ADDR_WIDTH{1'bx}};
161                     opcode_bit  = 3'bx;
162                     opcode_imd  = 12'bx;
163                 end
164                 16'b0010_10??_????_????: begin
165                     opcode_type = `TYPE_OR;
166                     opcode_rd   = instruction[8:4];
167                     opcode_rr   = {instruction[9], instruction[3:0]};
168                     opcode_bit  = 3'bx;
169                     opcode_imd  = 12'bx;
170                 end
171                 16'b1011_1???_????_????: begin
172                     opcode_type = `TYPE_OUT;
173                     opcode_rd   = {R_ADDR_WIDTH{1'bx}};
174                     opcode_rr   = instruction[8:4];
175                     opcode_bit  = 3'bx;
176                     opcode_imd  = {instruction[10:9], instruction[3:0]};
177                 end
178                 16'b1001_000?_????_1111: begin
179                     opcode_type = `TYPE_POP;
180                     opcode_rd   = instruction[8:4];
181                     opcode_rr   = {R_ADDR_WIDTH{1'bx}};
182                     opcode_bit  = 3'bx;
183                     opcode_imd  = 12'bx;
184                 end
185                 16'b1001_001?_????_1111: begin
186                     opcode_type = `TYPE_PUSH;
187                     opcode_rd   = {R_ADDR_WIDTH{1'bx}};
188                     opcode_rr   = instruction[8:4];
189                     opcode_bit  = 3'bx;
190                     opcode_imd  = 12'bx;
191                 end
192                 16'b1101_????_????_????: begin
193                     opcode_type = `TYPE_RCALL;
194                     opcode_rd   = {R_ADDR_WIDTH{1'bx}};
195                     opcode_rr   = {R_ADDR_WIDTH{1'bx}};
196                     opcode_bit  = 3'bx;
197                     opcode_imd  = instruction[11:0];
198                 end
199                 16'b1001_0101_0000_1000: begin
200                     opcode_type = `TYPE_RET;
201                     opcode_rd   = {R_ADDR_WIDTH{1'bx}};
202                     opcode_rr   = {R_ADDR_WIDTH{1'bx}};
203                     opcode_bit  = 3'bx;
204                     opcode_imd  = 12'bx;
205                 end
206                 16'b1001_0101_0001_1000: begin
207                     opcode_type = `TYPE_RETI;
```

```
                    opcode_type = `TYPE_RETI;
                    opcode_rd   = {R_ADDR_WIDTH{1'bx}};
                    opcode_rr   = {R_ADDR_WIDTH{1'bx}};
                    opcode_bit  = 3'bx;
                    opcode_imd  = 12'bx;
                end
                16'b1100_????_????_????: begin
                    opcode_type = `TYPE_RJMP;
                    opcode_rd   = {R_ADDR_WIDTH{1'bx}};
                    opcode_rr   = {R_ADDR_WIDTH{1'bx}};
                    opcode_bit  = 3'bx;
                    opcode_imd  = instruction[11:0];
                end
                16'b1001_1010_????_????: begin
                    opcode_type = `TYPE_SBI;
                    opcode_rd   = {R_ADDR_WIDTH{1'bx}};
                    opcode_rr   = {R_ADDR_WIDTH{1'bx}};
                    opcode_bit  = instruction[2:0];
                    opcode_imd  = {7'b0, instruction[7:3]};
                end
                16'b1001_0100_0111_1000: begin
                    opcode_type = `TYPE_SEI;
                    opcode_rd   = {R_ADDR_WIDTH{1'bx}};
                    opcode_rr   = {R_ADDR_WIDTH{1'bx}};
                    opcode_bit  = `FLAGS_I;
                    opcode_imd  = `SREG;
                end
                16'b0001_10??_????_????: begin
                    opcode_type = `TYPE_SUB;
                    opcode_rd   = instruction[8:4];
                    opcode_rr   = {instruction[9], instruction[3:0]};
                    opcode_bit  = 3'bx;
                    opcode_imd  = 12'bx;
                end
                16'b0101_????_????_????: begin
                    opcode_type = `TYPE_SUBI;
                    opcode_rd   = {1'b1, instruction[7:4]};
                    opcode_rr   = {R_ADDR_WIDTH{1'bx}};
                    opcode_bit  = 3'bx;
                    opcode_imd  = {4'b0, instruction[7:4], instruction[3:0]};
                end
                16'b1010_1???_????_????: begin
                    opcode_type = `TYPE_STS;
                    opcode_rd   = {R_ADDR_WIDTH{1'bx}};
                    opcode_rr   = {1'b1, instruction[7:4]};
                    opcode_bit  = 3'bx;
                    opcode_imd  = {~instruction[8], instruction[8],
                        instruction[10:9], instruction[3:0]};
                end
                default: begin
                    opcode_type = `TYPE_UNKNOWN;
                    opcode_rd   = {R_ADDR_WIDTH{1'bx}};
                    opcode_rr   = {R_ADDR_WIDTH{1'bx}};
                    opcode_bit  = 3'bx;
                    opcode_imd  = 12'bx;
                end
            endcase
        end
    end

    /* Opcode groups */
    // TODO 2: add the given instruction to the right groups
    // It needs to execute arithmetic operations on ALU
    assign opcode_group[`GROUP_ALU_AUX] =
        (opcode_type == `TYPE_SBI) ||
        (opcode_type == `TYPE_CBI) ||
        (opcode_type == `TYPE_SEI) ||
        (opcode_type == `TYPE_CLI) ||
        (opcode_type == `TYPE_RETI)||
        (opcode_type == `TYPE_CALL_ISR);
    assign opcode_group[`GROUP_ALU_IMD] =
        (opcode_type == `TYPE_INC) ||
        (opcode_type == `TYPE_DEC) ||
        (opcode_type == `TYPE_CPI);
    assign opcode_group[`GROUP_ALU_ONE_OP] =
        (opcode_type == `TYPE_NEG);
    assign opcode_group[`GROUP_ALU_TWO_OP] =
        (opcode_type == `TYPE_ADD) ||
        (opcode_type == `TYPE_ADC) ||
        (opcode_type == `TYPE_SUB) ||
        (opcode_type == `TYPE_AND) ||
        (opcode_type == `TYPE_EOR) ||
        (opcode_type == `TYPE_OR)  ||
        (opcode_type == `TYPE_CP);
    assign opcode_group[`GROUP_ALU] =
        opcode_group[`GROUP_ALU_ONE_OP] ||
        opcode_group[`GROUP_ALU_TWO_OP];

    assign opcode_group[`GROUP_LOAD_DIRECT] =
        (opcode_type == `TYPE_LDS);

    assign opcode_group[`GROUP_LOAD_INDIRECT] =
        (opcode_type == `TYPE_LD_Y) ||
        (opcode_type == `TYPE_POP) ||
        (opcode_type == `TYPE_RET) ||
        (opcode_type == `TYPE_RETI);
    assign opcode_group[`GROUP_LOAD] =
        opcode_group[`GROUP_LOAD_DIRECT] ||
        opcode_group[`GROUP_LOAD_INDIRECT];

    assign opcode_group[`GROUP_STORE_DIRECT] =
        (opcode_type == `TYPE_STS);

    assign opcode_group[`GROUP_STORE_INDIRECT] =
```

```
311           (opcode_type == `TYPE_PUSH)  ||
312           (opcode_type == `TYPE_RCALL) ||
313           (opcode_type == `TYPE_CALL_ISR);
314       assign opcode_group[`GROUP_STORE] =
315           opcode_group[`GROUP_STORE_DIRECT] ||
316           opcode_group[`GROUP_STORE_INDIRECT];
317
318       assign opcode_group[`GROUP_STACK] =
319           (opcode_type == `TYPE_PUSH)     ||
320           (opcode_type == `TYPE_POP)      ||
321           (opcode_type == `TYPE_RCALL)    ||
322           (opcode_type == `TYPE_CALL_ISR) ||
323           (opcode_type == `TYPE_RETI)     ||
324           (opcode_type == `TYPE_RET);
325
326       assign opcode_group[`GROUP_MEMORY] =
327           (opcode_group[`GROUP_LOAD] ||
328            opcode_group[`GROUP_STORE]);
329
330       assign opcode_group[`GROUP_REGISTER] =
331           (opcode_type == `TYPE_LDI) ||
332           (opcode_type == `TYPE_MOV);
333
334       assign opcode_group[`GROUP_CONTROL_FLOW] =
335           (opcode_type == `TYPE_BRBS)  ||
336           (opcode_type == `TYPE_BRBC)  ||
337           (opcode_type == `TYPE_RJMP)  ||
338           (opcode_type == `TYPE_RCALL) ||
339           (opcode_type == `TYPE_RET)   ||
340           (opcode_type == `TYPE_CALL_ISR);
341
342       assign opcode_group[`GROUP_IO_READ] =
343           (opcode_type  == `TYPE_IN)  ||   // access any I/O address
344           (opcode_type  == `TYPE_ADC) ||   // access SREG
345           (opcode_group[`GROUP_STACK])||   // access SP
346           (opcode_group[`GROUP_ALU_AUX]);  // read value to modify from I/O space
347       assign opcode_group[`GROUP_IO_WRITE] =
348           (opcode_type  == `TYPE_OUT) ||   // access any I/O address
349           (opcode_group[`GROUP_ALU])  ||   // access SREG
350           (opcode_group[`GROUP_STACK])||   // access SP
351           (opcode_group[`GROUP_ALU_AUX]);  // write modified value to I/O space
352
353       assign opcode_group[`GROUP_TWO_CYCLE_MEM] =
354           (opcode_type == `TYPE_RCALL)    ||
355           (opcode_type == `TYPE_CALL_ISR) ||
356           (opcode_type == `TYPE_RETI)     ||
357           (opcode_type == `TYPE_RET);
358
359
360       assign opcode_group[`GROUP_TWO_CYCLE_WB] =
361           (opcode_type == `TYPE_CALL_ISR) ||
362           (opcode_type == `TYPE_RETI);
363
364       assign opcode_group[`GROUP_TWO_CYCLE_ID] =
365           (opcode_type == `TYPE_CALL_ISR) ||
366           (opcode_type == `TYPE_RETI);
367
368   endmodule
369
```

defines.vh

```
/* In order to disable implicit declaration of wires */
`default_nettype none

/* SREG flags */
`define FLAGS_C             3'd0
`define FLAGS_Z             3'd1
`define FLAGS_N             3'd2
`define FLAGS_V             3'd3
`define FLAGS_S             3'd4
`define FLAGS_H             3'd5
`define FLAGS_T             3'd6
`define FLAGS_I             3'd7

/* Indirect registers */
`define XL                  5'd26
`define XH                  5'd27
`define YL                  5'd28
`define YH                  5'd29
`define ZL                  5'd30
`define ZH                  5'd31

/* IO addresses */
`define SREG                6'h3F
`define SPH                 6'h3E
`define SPL                 6'h3D
`define TIMSK               6'h26
`define TIFR                6'h25
`define TCCR0A              6'h19
`define TCCR0B              6'h18
`define TCNT0               6'h17
`define OCR0A               6'h16
`define OCR0B               6'h15
`define PUEB                6'h07
`define PORTB               6'h06
`define DDRB                6'h05
`define PINB                6'h04
`define PUEA                6'h03
`define PORTA               6'h02
`define DDRA                6'h01
`define PINA                6'h00

/* Interrupt enable flags */
`define OCIE0B              3'd2
`define OCIE0A              3'd1
`define TOIE0               3'd0

/* Interrupt flags */
`define OCF0B               3'd2
`define OCF0A               3'd1
`define TOV0                3'd0

/* Timer modes */
`define INVALID             3'd0
`define NORMAL              3'd1
`define CTC                 3'd2
`define FAST_PWM_MAX        3'd3
`define FAST_PWM_OCR        3'd4

/* Interrupt vectors for ISRs */
`define TIM0_COMPA_ISR      10'h09
`define TIM0_COMPB_ISR      10'h0A
`define TIM0_OVF_ISR        10'h0B

/* States for FSM */
`define STATE_IF            1
`define STATE_ID            2
`define STATE_EX            3
`define STATE_MEM           4
`define STATE_WB            5
`define STATE_RESET         0
`define STATE_COUNT         6

/* Opcode types deduced by decoder */
`define TYPE_NOP            0
`define TYPE_UNKNOWN        1
`define TYPE_LD_X           2
`define TYPE_LD_X_POSTINC   3
`define TYPE_LD_X_PREDEC    4
`define TYPE_LD_Y           5
`define TYPE_LD_Y_POSTINC   6
`define TYPE_LD_Y_PREDEC    7
`define TYPE_LD_Z           8
`define TYPE_LD_Z_POSTINC   9
`define TYPE_LD_Z_PREDEC    10
`define TYPE_LDS            11
`define TYPE_ST_X           12
`define TYPE_ST_X_POSTINC   13
`define TYPE_ST_X_PREDEC    14
`define TYPE_ST_Y           15
`define TYPE_ST_Y_POSTINC   16
`define TYPE_ST_Y_PREDEC    17
`define TYPE_ST_Z           18
`define TYPE_ST_Z_POSTINC   19
`define TYPE_ST_Z_PREDEC    20
`define TYPE_STS            21
`define TYPE_LDI            22
`define TYPE_MOV            23
`define TYPE_BRBS           24
`define TYPE_BRBC           25
`define TYPE_RJMP           26
`define TYPE_RCALL          27
`define TYPE_RET            28
`define TYPE_POP            29
```

```
104   `define TYPE_PUSH           30
105   `define TYPE_IN             31
106   `define TYPE_OUT            32
107   `define TYPE_SBI            33
108   `define TYPE_CBI            34
109   `define TYPE_INC            35
110   `define TYPE_DEC            36
111   `define TYPE_ADD            37
112   `define TYPE_ADC            38
113   `define TYPE_SUB            39
114   `define TYPE_SUBI           40
115   `define TYPE_SBC            41
116   `define TYPE_SBCI           42
117   `define TYPE_CP             43
118   `define TYPE_CPI            44
119   `define TYPE_NEG            45
120   `define TYPE_AND            46
121   `define TYPE_ANDI           47
122   `define TYPE_OR             48
123   `define TYPE_ORI            49
124   `define TYPE_EOR            50
125   `define TYPE_WDR            51
126   `define TYPE_SEI            52
127   `define TYPE_CLI            53
128   `define TYPE_CALL_ISR       54
129   `define TYPE_RETI           55
130   `define TYPE_SOP            56
131   `define TYPE_LSP_Y          57
132   `define TYPE_SQUARE_SUM     58
133   `define TYPE_CUBE_SUM       59
134   `define OPCODE_COUNT        6
135
136   /* Opcode groups */
137   `define GROUP_ALU             0
138   `define GROUP_ALU_ONE_OP      1
139   `define GROUP_ALU_TWO_OP      2
140   `define GROUP_ALU_AUX         3
141   `define GROUP_ALU_IMD         4
142   `define GROUP_REGISTER        5
143   `define GROUP_LOAD_INDIRECT   6
144   `define GROUP_LOAD_DIRECT     7
145   `define GROUP_LOAD            8
146   `define GROUP_STORE_INDIRECT  9
147   `define GROUP_STORE_DIRECT    10
148   `define GROUP_STORE           11
149   `define GROUP_STACK           12
150   `define GROUP_MEMORY          13
151   `define GROUP_CONTROL_FLOW    14
152   `define GROUP_IO_READ         15
153   `define GROUP_IO_WRITE        16
154   `define GROUP_TWO_CYCLE_MEM   17
155   `define GROUP_TWO_CYCLE_WB    18
156   `define GROUP_TWO_CYCLE_ID    19
157   //`define GROUP_TWO_CYCLE_EX  20
158   //`define GROUP_TWO_CYCLE_IF  21
159   `define GROUP_COUNT           20
160
161   /* Operations permitted by ALU */
162   `define OPSEL_NONE          0
163   `define OPSEL_ADD           1
164   `define OPSEL_ADC           2
165   `define OPSEL_SUB           3
166   `define OPSEL_AND           4
167   `define OPSEL_XOR           5
168   `define OPSEL_OR            6
169   `define OPSEL_NEG           7
170   `define OPSEL_SQUARE_SUM    8
171   `define OPSEL_CUBE_SUM      9
172   `define OPSEL_COUNT         4
173
174   /* Control signals */
175   `define CONTROL_MEM_READ        0
176   `define CONTROL_MEM_WRITE       1
177   `define CONTROL_REG_RR_READ     2
178   `define CONTROL_REG_RR_WRITE    3
179   `define CONTROL_REG_RD_READ     4
180   `define CONTROL_REG_RD_WRITE    5
181   `define CONTROL_IO_READ         6
182   `define CONTROL_IO_WRITE        7
183   `define CONTROL_PREINC          8
184   `define CONTROL_POSTDEC         9
185   `define SIGNAL_COUNT            10
186
```

gpio.v

```
`include "defines.vh"
module io_sram #(
        parameter   DATA_WIDTH = 8,
        parameter   ADDR_WIDTH = 6,  // 64 I/O registers
        parameter I_ADDR_WIDTH = 10  // 1024 valid instructions
    )(
        // to host cpu
        input  wire                    clk,
        input  wire                    reset,
        input  wire                    oe,
        input  wire                    cs,
        input  wire                    we,
        input  wire   [ADDR_WIDTH-1:0] address,
        inout  wire   [DATA_WIDTH-1:0] data,
        // to external pins
        inout  wire   [DATA_WIDTH-1:0] pa,
        inout  wire   [DATA_WIDTH-1:0] pb,
        output wire                    oc0a,
        output wire                    oc0b,
        input  wire                    ack,
        output wire                    irq,
        output wire [I_ADDR_WIDTH-1:0] vector
    );

    reg  [DATA_WIDTH-1:0] memory[0:(1<<ADDR_WIDTH)-1];
    reg  [ADDR_WIDTH-1:0] addr_buf;
    wire                  readonly;
    wire [DATA_WIDTH-1:0] pa_buf;
    wire [DATA_WIDTH-1:0] pb_buf;
    wire [DATA_WIDTH-1:0] tcnt0;
    wire [DATA_WIDTH-1:0] tifr;
    reg    [ADDR_WIDTH:0] i;

    always @(negedge clk)
    begin
        if (reset) begin
            for (i = 0; i < (1<<ADDR_WIDTH); i = i + 1) begin
                if(i==`SPL) begin
                    memory[i] <= 8'hbf;
                end else begin
                    memory[i] <= 0;
                end
            end
        end
        else if (cs) begin
            if (we && !readonly) begin
                memory[address] <= data;
            end else begin
                addr_buf <= address;
            end
        end
        memory[`PINA]  <= pa;     // taken directly from external pin
        memory[`PINB]  <= pb;     // taken directly from external pin
        // resolve write conflict between CPU and timer0,
        // give priority to CPU
        if (address == `TCNT0 && cs && we)
            memory[`TCNT0] <= data;   // cpu allowed to change counter
        else
            memory[`TCNT0] <= tcnt0;  // increment timer0 here
        if (address == `TIFR && cs && we)
            memory[`TIFR]  <= data;
        else begin
            if (ack) begin
                case (vector)
                    `TIM0_OVF_ISR: begin
                        memory[`TIFR] <= tifr & ~(1 << `TOV0);
                    end
                    `TIM0_COMPA_ISR: begin
                        memory[`TIFR] <= tifr & ~(1 << `OCF0A);
                    end
                    `TIM0_COMPB_ISR: begin
                        memory[`TIFR] <= tifr & ~(1 << `OCF0B);
                    end
                endcase
            end else begin
                memory[`TIFR] <= tifr;
            end
        end
    end

    assign data = (cs && oe && !we) ? memory[addr_buf] : {DATA_WIDTH{1'bz}};
    assign readonly = (address == `PINA || address == `PINB);

    assign pa = pa_buf;    // directly to external pin
    assign pb = pb_buf;    // directly to external pin

    gpio_unit #(
        .DATA_WIDTH(DATA_WIDTH)
    ) gpio (
        .clk      (clk),
        .reset    (reset),
        .mem_ddra (memory[`DDRA]),
        .mem_ddrb (memory[`DDRB]),
        .mem_porta(memory[`PORTA]),
        .mem_portb(memory[`PORTB]),
        .pa_buf   (pa_buf),
        .pb_buf   (pb_buf)
    );

    timer_unit #(
        .DATA_WIDTH(DATA_WIDTH)
    ) timer0 (
        .clk      (clk),
```

```
104          .reset     (reset),
105          .mem_tifr  (memory[`TIFR]),
106          .mem_tcnt0 (memory[`TCNT0]),
107          .mem_tccr0a(memory[`TCCR0A]),
108          .mem_tccr0b(memory[`TCCR0B]),
109          .mem_ocr0a (memory[`OCR0A]),
110          .mem_ocr0b (memory[`OCR0B]),
111          .tcnt0     (tcnt0),
112          .tifr      (tifr),
113          .oc0a      (oc0a),
114          .oc0b      (oc0b)
115      );
116
117      interrupt_controller #(
118          .DATA_WIDTH(DATA_WIDTH),
119          .I_ADDR_WIDTH(I_ADDR_WIDTH)
120      ) pic (
121          .clk      (clk),
122          .reset    (reset),
123          .mem_tifr (memory[`TIFR]),
124          .mem_timsk(memory[`TIMSK]),
125          .mem_sreg (memory[`SREG]),
126          .irq      (irq),
127          .vector   (vector)
128      );
129  endmodule
130
```

gpio_unit.v

```
1   `include "defines.vh"
2   module gpio_unit #(
3           parameter   DATA_WIDTH = 8
4       )(
5           // host interface
6           input wire                  clk,
7           input wire                  reset,
8           input wire [DATA_WIDTH-1:0] mem_ddra,
9           input wire [DATA_WIDTH-1:0] mem_ddrb,
10          input wire [DATA_WIDTH-1:0] mem_porta,
11          input wire [DATA_WIDTH-1:0] mem_portb,
12          // output buffers for GPIO external pins
13          output reg [DATA_WIDTH-1:0] pa_buf,
14          output reg [DATA_WIDTH-1:0] pb_buf
15      );
16
17      reg [DATA_WIDTH:0] i;
18      always @(posedge clk) begin
19          if (reset)
20          begin
21              pa_buf    <= 0;
22              pb_buf    <= 0;
23          end
24          else
25          for (i = 0; i < DATA_WIDTH; i = i + 1) begin : gen_ports
26              pa_buf[i] <= mem_ddra[i] ? mem_porta[i] : 1'bz;
27              pb_buf[i] <= mem_ddrb[i] ? mem_portb[i] : 1'bz;
28          end
29      end
30
31  endmodule
32
```

interrupt_controller.v

```
1   `include "defines.vh"
2   module interrupt_controller #(
3           parameter   DATA_WIDTH = 8,
4           parameter I_ADDR_WIDTH = 10
5       )(
6           input wire                    clk,
7           input wire                    reset,
8           input wire   [DATA_WIDTH-1:0] mem_tifr,
9           input wire   [DATA_WIDTH-1:0] mem_timsk,
10          input wire   [DATA_WIDTH-1:0] mem_sreg,
11          output reg                    irq,
12          output reg [I_ADDR_WIDTH-1:0] vector
13      );
14
15      always @(posedge clk) begin
16          if (reset) begin
17              irq    <= 0;
18              vector <= 0;
19          end else begin
20              if (mem_sreg[`FLAGS_I] && mem_timsk[`OCIE0A] && mem_tifr[`OCF0A]) begin
21                  irq    <= 1;
22                  vector <= `TIM0_COMPA_ISR;
23              end else if (mem_sreg[`FLAGS_I] && mem_timsk[`OCIE0B] && mem_tifr[`OCF0B]) begin
24                  irq    <= 1;
25                  vector <= `TIM0_COMPB_ISR;
26              end else if (mem_sreg[`FLAGS_I] && mem_timsk[`TOIE0] && mem_tifr[`TOV0]) begin
27                  irq    <= 1;
28                  vector <= `TIM0_OVF_ISR;
29              end else begin
30                  irq    <= 0;
31              end
32          end
33      end
34  endmodule
35
```

reg_file_interface_unit.v

```
`include "defines.vh"
module reg_file_interface_unit #(
        parameter INSTR_WIDTH  = 16,   // instructions are 16 bits in width
        parameter DATA_WIDTH   = 8,    // registers are 8 bits in width
        parameter D_ADDR_WIDTH = 8,    // 256 bytes address space
        parameter R_ADDR_WIDTH = 5     // 32 registers
    )(
        input  wire [`OPCODE_COUNT-1:0] opcode_type,
        input  wire  [R_ADDR_WIDTH-1:0] opcode_rd,
        input  wire  [R_ADDR_WIDTH-1:0] opcode_rr,
        input  wire [`SIGNAL_COUNT-1:0] signals,
        input  wire                     cycle_count,
        input  wire    [DATA_WIDTH-1:0] writeback_value,
        output wire  [R_ADDR_WIDTH-1:0] rr_addr,
        output wire  [R_ADDR_WIDTH-1:0] rd_addr,
        inout  wire    [DATA_WIDTH-1:0] rr_data,
        inout  wire    [DATA_WIDTH-1:0] rd_data,
        output wire                     rr_cs,
        output wire                     rd_cs,
        output wire                     rr_we,
        output wire                     rd_we,
        output wire                     rr_oe,
        output wire                     rd_oe
    );

    wire [R_ADDR_WIDTH-1:0] internal_rr_addr;
    wire [R_ADDR_WIDTH-1:0] internal_rd_addr;

    /* Internal */
    assign internal_rd_addr =
            signals[`CONTROL_REG_RD_READ] ?
                (opcode_type == `TYPE_LD_X || opcode_type == `TYPE_ST_X) ? `XL :
                (opcode_type == `TYPE_LD_Y || opcode_type == `TYPE_ST_Y) ? `YL :
                (opcode_type == `TYPE_LD_Z || opcode_type == `TYPE_ST_Z) ? `ZL :
                opcode_rd :
            signals[`CONTROL_REG_RD_WRITE] ?
                opcode_rd :
            {R_ADDR_WIDTH{1'bx}};
    assign internal_rr_addr =
            signals[`CONTROL_REG_RR_READ] ?
                (opcode_type == `TYPE_LD_X || opcode_type == `TYPE_ST_X) ? `XH :
                (opcode_type == `TYPE_LD_Y || opcode_type == `TYPE_ST_Y) ? `YH :
                (opcode_type == `TYPE_LD_Z || opcode_type == `TYPE_ST_Z) ? `ZH :
                 opcode_rr :
            {R_ADDR_WIDTH{1'bx}};

    assign rd_data = signals[`CONTROL_REG_RD_WRITE] ?
                     writeback_value :
                     {DATA_WIDTH{1'bz}};
    assign rr_data = {DATA_WIDTH{1'bz}};
    assign rd_we   = signals[`CONTROL_REG_RD_READ]  ? 1'b0 :
                     signals[`CONTROL_REG_RD_WRITE] ? 1'b1 : 1'bx;
    assign rr_we   = signals[`CONTROL_REG_RR_READ]  ? 1'b0 : 1'bx;
    assign rd_oe   = signals[`CONTROL_REG_RD_READ]  ? 1'b1 : 1'bx;
    assign rr_oe   = signals[`CONTROL_REG_RR_READ]  ? 1'b1 : 1'bx;
    assign rd_cs   = signals[`CONTROL_REG_RD_READ] ||
                     signals[`CONTROL_REG_RD_WRITE];
    assign rr_cs   = signals[`CONTROL_REG_RR_READ];
    assign rd_addr = (rd_cs) ? internal_rd_addr : {R_ADDR_WIDTH{1'bx}};
    assign rr_addr = (rr_cs) ? internal_rr_addr : {R_ADDR_WIDTH{1'bx}};

endmodule

```

register_file.v

```
/* Traducerea si adaptarea:
 * http://www.asic-world.com/examples/verilog/ram_dp_sr_sw.html
 */
module dual_port_sram #(
        parameter DATA_WIDTH = 8,    // registers are 8 bits in width
        parameter ADDR_WIDTH = 5     // 32 registers
    )(
        input  wire                  clk,
        input  wire [ADDR_WIDTH-1:0] rr_addr,
        input  wire [ADDR_WIDTH-1:0] rd_addr,
        inout  wire [DATA_WIDTH-1:0] rr_data,
        inout  wire [DATA_WIDTH-1:0] rd_data,
        input  wire                  rr_cs,
        input  wire                  rd_cs,
        input  wire                  rr_we,
        input  wire                  rd_we,
        input  wire                  rr_oe,
        input  wire                  rd_oe
    );

    reg [DATA_WIDTH-1:0] mem[0:(1<<ADDR_WIDTH)-1];
    reg [ADDR_WIDTH-1:0] rr_addr_buf;
    reg [ADDR_WIDTH-1:0] rd_addr_buf;

    reg [ADDR_WIDTH:0] i;
    initial begin
        for (i = 0; i < (1<<ADDR_WIDTH); i = i + 1)
            mem[i] <= 0;
    end

    /* Memory Write Block
     * Write Operation : When we = 1, cs = 1
     */
    always @(negedge clk) begin : MEM_WRITE
        if (rr_cs && rr_we) begin
            mem[rr_addr] <= rr_data;
        end else if (rd_cs && rd_we) begin
            mem[rd_addr] <= rd_data;
        end
    end

    /* Memory Read Block
     * Read Operation : When we = 0, cs = 1
     */
    always @(negedge clk) begin : MEM_READ
        /* First port of RAM */
        if (rr_cs && !rr_we) begin
            rr_addr_buf <= rr_addr;
        end
        /* Second port of RAM */
        if (rd_cs && !rd_we) begin
            rd_addr_buf <= rd_addr;
        end
    end

    /* Tri-State Buffer control
     * output : When we = 0, oe = 1, cs = 1
     */
    assign rr_data = (rr_cs && rr_oe && !rr_we) ? mem[rr_addr_buf] : {DATA_WIDTH{1'bz}};
    assign rd_data = (rd_cs && rd_oe && !rd_we) ? mem[rd_addr_buf] : {DATA_WIDTH{1'bz}};

endmodule

```

rom.v

```
module rom #(
        parameter   DATA_WIDTH = 16,
        parameter   ADDR_WIDTH = 8          // 2 * 1024 bytes of ROM
    )(
        input  wire                  clk,
        input  wire [ADDR_WIDTH-1:0] addr,  // here comes the program counter
        output  reg [DATA_WIDTH-1:0] data   // here goes the instruction
    );

    reg [DATA_WIDTH-1:0] value;
    //DO NOT MODIFY
    always @* begin
        case (addr)
        /*   ldi    r16, 5      */
        0:  value = 16'b1110_0000_0000_0101;

        /*   ldi    r17, 6      */
        1:  value = 16'b1110_0000_0001_0110;

        /*   ldi    r18, 4      */
        2:  value = 16'b1110_0000_0010_0100;

        /*   ldi    r19, 2      */
        3:  value = 16'b1110_0000_0011_0010;

        /*   square_sum     r16 , r17       */
        4:  value = 16'b0011_1010_0001_0001;

        /*   cube_sum   r18 , r19       */
        5:  value = 16'b0111_1011_0010_0011;

      default:    value = 16'b0000000000000000;
        endcase
    end

    always @(negedge clk) begin
        data <= value;
    end

endmodule

```

signal_generation_unit.v

```
`include "defines.vh"
module signal_generation_unit (
        input  wire  [`STATE_COUNT-1:0] state,
        input  wire                     cycle_count,
        input  wire [`OPCODE_COUNT-1:0] opcode_type,
        input  wire  [`GROUP_COUNT-1:0] opcode_group,
        output wire [`SIGNAL_COUNT-1:0] signals
    );

    /* Control signals */

    /* Register interface logic */
    assign signals[`CONTROL_REG_RR_READ] =
            (state == `STATE_ID) &&
            (opcode_group[`GROUP_ALU_TWO_OP] ||
             opcode_group[`GROUP_LOAD_INDIRECT]  ||
             opcode_group[`GROUP_STORE] ||
             opcode_type == `TYPE_MOV ||
             opcode_type == `TYPE_OUT);
    assign signals[`CONTROL_REG_RR_WRITE] = 0;
    assign signals[`CONTROL_REG_RD_READ] =
            (state == `STATE_ID) &&
            (opcode_group[`GROUP_ALU] ||
             opcode_group[`GROUP_ALU_IMD] ||
            ((opcode_group[`GROUP_STORE_INDIRECT] ||
              opcode_group[`GROUP_LOAD_INDIRECT]) &&
             !opcode_group[`GROUP_STACK]));

    assign signals[`CONTROL_REG_RD_WRITE] =
            (state == `STATE_WB) &&
            (opcode_group[`GROUP_REGISTER] ||
            (opcode_group[`GROUP_ALU]  && opcode_type != `TYPE_CP)  ||
            (opcode_group[`GROUP_ALU_IMD] && opcode_type != `TYPE_CPI) ||
            (opcode_group[`GROUP_LOAD] && opcode_type != `TYPE_RET && opcode_type != `TYPE_RETI) ||
             opcode_type == `TYPE_IN);

    assign signals[`CONTROL_MEM_READ] =
           (state == `STATE_MEM) &&
           opcode_group[`GROUP_LOAD];
    assign signals[`CONTROL_MEM_WRITE] =
           (state == `STATE_MEM) &&
           opcode_group[`GROUP_STORE];

    assign signals[`CONTROL_IO_READ] =
        (state == `STATE_ID) &&
        opcode_group[`GROUP_IO_READ];
    assign signals[`CONTROL_IO_WRITE] =
        (state == `STATE_WB) &&
        opcode_group[`GROUP_IO_WRITE];

     assign signals[`CONTROL_POSTDEC] =
            (opcode_type == `TYPE_PUSH  ||
             opcode_type == `TYPE_RCALL ||
             opcode_type == `TYPE_CALL_ISR) &&
            (state == `STATE_MEM);
    assign signals[`CONTROL_PREINC] =
            (opcode_type == `TYPE_POP) ?
                (state == `STATE_EX) :
            (opcode_type == `TYPE_RET || opcode_type == `TYPE_RETI) &&
                (state == `STATE_EX || (state == `STATE_MEM && cycle_count == 0));

endmodule

```

sram.v

```
`include "defines.vh"
module sram #(
        parameter DATA_WIDTH  = 8,
        parameter ADDR_WIDTH  = 8       // 256 bytes address space
    )(
        input  wire                  clk,
        input  wire                  oe,
        input  wire                  cs,
        input  wire                  we,
        input  wire [ADDR_WIDTH-1:0] address,
        inout  wire [DATA_WIDTH-1:0] data
    );

    reg [DATA_WIDTH-1:0] memory[0:(1<<ADDR_WIDTH)-1];
    reg [ADDR_WIDTH-1:0] addr_buf;

    reg [ADDR_WIDTH:0] i;
    initial begin
        for (i = 0; i < (1<<ADDR_WIDTH); i = i + 1) begin
            memory[i] <= 0;
        end
    end

    always @(negedge clk)
    begin
        if (cs) begin
            if (we) begin
                memory[address] <= data;
            end else begin
                addr_buf <= address;
            end
        end
    end

    assign data = (cs && oe && !we) ? memory[addr_buf] : {DATA_WIDTH{1'bz}};

endmodule

```

state_machine.v

```
`include "defines.vh"
module state_machine (
        output reg  [`STATE_COUNT-1:0] state,
        output reg                     cycle_count,
        input  wire [`GROUP_COUNT-1:0] opcode_group,
        input  wire                    clk,
        input  wire                    reset
    );

    reg [`STATE_COUNT-1:0] next_state;

    always @(posedge clk, posedge reset)
        if (reset)
            state <= `STATE_RESET;
        else
            state <= next_state;

    always @(posedge clk, posedge reset) begin
        if (reset)
            cycle_count <= 0;
        else if (state == `STATE_MEM &&
                 opcode_group[`GROUP_TWO_CYCLE_MEM])
            cycle_count <= ~cycle_count;
        else if (state == `STATE_WB &&
                 opcode_group[`GROUP_TWO_CYCLE_WB])
            cycle_count <= ~cycle_count;
        else if (state == `STATE_ID &&
                 opcode_group[`GROUP_TWO_CYCLE_ID])
            cycle_count <= ~cycle_count;
    end

    always @* begin
        case (state)
        `STATE_RESET:
            next_state = `STATE_IF;
        `STATE_IF:
            next_state = `STATE_ID;
        `STATE_ID:
            if (opcode_group[`GROUP_TWO_CYCLE_ID] && cycle_count == 0)
                next_state = `STATE_ID;
            else
                next_state = `STATE_EX;
        `STATE_EX:
            next_state = `STATE_MEM;
        `STATE_MEM:
            if (opcode_group[`GROUP_TWO_CYCLE_MEM] && cycle_count == 0)
                next_state = `STATE_MEM;
            else
                next_state = `STATE_WB;
        `STATE_WB:
            if (opcode_group[`GROUP_TWO_CYCLE_WB] && cycle_count == 0)
                next_state = `STATE_WB;
            else
                next_state = `STATE_IF;
        /* Should never get here, but anyway */
        default:
            next_state = `STATE_RESET;
        endcase
    end

endmodule

```

timer_unit.v

```
`include "defines.vh"
module timer_unit #(
        parameter    DATA_WIDTH = 8
    )(
        input  wire                  clk,        // input clock
        input  wire                  reset,      // reset signal
        input  wire [DATA_WIDTH-1:0] mem_ocr0a,  // OCR0A / OCR0B (Output Compare Register 0, channel A/B)
        input  wire [DATA_WIDTH-1:0] mem_ocr0b,  // OCR0A / OCR0B (Output Compare Register 0, channel A/B)
        input  wire [DATA_WIDTH-1:0] mem_tcnt0,  // value from memory of counter for timer 0 (8 bits)
        input  wire [DATA_WIDTH-1:0] mem_tifr,   // TIFR - Timer/Counter Interrupt Flag Register
        input  wire [DATA_WIDTH-1:0] mem_tccr0a, // TCCR0A, TCCR0B (Timer/Counter Control Registers A and B)
        input  wire [DATA_WIDTH-1:0] mem_tccr0b, // TCCR0A, TCCR0B (Timer/Counter Control Registers A and B)
        output reg  [DATA_WIDTH-1:0] tcnt0,      // next value of counter
        output reg  [DATA_WIDTH-1:0] tifr,       // next tifr
        output reg                   oc0a,       // output pin channel A
        output reg                   oc0b        // output pin channel B
    );

    // Signals used for prescaling
    wire    clk_t;
    wire    clk_io,      // original clk signal
            clk_io_8,    // clk prescaled with 8    (8   times slower)
            clk_io_64,   // clk prescaled with 64   (64  times slower)
            clk_io_256,  // clk prescaled with 256  (256 times slower)
            clk_io_1024; // clk prescaled with 1024 (1024 times slower)

    wire     [7:0] clk_sources;  // array of clk signals
    reg     [13:0] count;        // helper for prescaler

    always @(posedge clk, posedge reset) begin
        if (reset)
            count <= 0;
        else
            count <= count + 1;
    end

    assign clk_io           = clk;
    assign clk_io_8         = count[2]; // clk / 2^3
    assign clk_io_64        = count[5]; // clk / 2^6
    assign clk_io_256       = count[7]; // clk / 2^8
    assign clk_io_1024      = count[9]; // clk / 2^10

    assign clk_sources = {
        1'b0, // cs0 == 7 - We don't support this options, so don't change this
        1'b0, // cs0 == 6 - We don't support this options, so don't change this
        clk_io_1024,  // cs0 == 5 - clkI/O / 1024   (From prescaler)
        clk_io_256,   // cs0 == 4 - clkI/O / 256    (From prescaler)
        clk_io_64,    // cs0 == 3 - clkI/O / 64     (From prescaler)
        clk_io_8,     // cs0 == 2 - clkI/O / 8      (From prescaler)
        clk_io,       // cs0 == 1 - clkI/O          (No prescaling)
        1'b0          // cs0 == 0 - No clock source (Timer/Counter stopped)
        };

    assign clk_t = clk_sources[cs0];

    // Bits in configuration register
    wire            [1:0] com0a; // COM0A[1:0]: Compare Match Output A Mode
    wire            [1:0] com0b; // COM0B[1:0]: Compare Match Output B Mode
    wire            [2:0] wgm0;  // WGM0 [2:0]: Waveform Generation Mode
    wire            [2:0] cs0;   // CS0[2:0]: Clock Select

    assign com0a =  mem_tccr0a[7:6];                 // datasheet pages: 69
    assign com0b =  mem_tccr0a[5:4];                 // datasheet pages: 69
    assign wgm0  =  {mem_tccr0b[3], mem_tccr0a[1:0]}; // datasheet pages: 69, 72
    assign cs0   =  mem_tccr0b[2:0];                 // datasheet pages: 72

    wire [5:0]             timer_mode;  // Timer operation mode
    wire [DATA_WIDTH-1:0]  top;         // Value at which counter resets

    assign timer_mode =
        (wgm0 == 0) ? `NORMAL :
        (wgm0 == 2) ? `CTC :
        (wgm0 == 3) ? `FAST_PWM_MAX :
        (wgm0 == 7) ? `FAST_PWM_OCR :
        `INVALID;

    assign top =
        (timer_mode == `NORMAL) ? 8'hFF :
        (timer_mode == `CTC) ? mem_ocr0a :
        (timer_mode == `FAST_PWM_MAX) ? 8'hFF :
        (timer_mode == `FAST_PWM_OCR) ? mem_ocr0a :
        0;

    // Counter assignment block.
    always @(posedge clk_t, posedge reset) begin
        if (reset) begin
            tcnt0 <= 0;
        end else begin
            if (mem_tcnt0 == top) begin
                tcnt0 <= 0;
            end else begin
                tcnt0 <= mem_tcnt0 + 1;
            end
        end
    end

    // Interrupt flag assignment block.
    always @(posedge clk_t, posedge reset) begin
        if (reset) begin
            tifr  <= 0;
        end else begin
            tifr <= mem_tifr;

```

```
104             if (mem_tcnt0 == top && (timer_mode !=  CTC || mem_tcnt0 == 8'hFF)) begin
105                 tifr  <= mem_tifr | (1 << `TOV0);
106             end
107
108             if (mem_tcnt0 == mem_ocr0a) begin
109                 tifr  <= mem_tifr | (1 << `OCF0A);
110             end
111
112             if (mem_tcnt0 == mem_ocr0b) begin
113                 tifr  <= mem_tifr | (1 << `OCF0B);
114             end
115         end
116     end
117
118     // OC0A, OC0B assignment block.
119     always @(posedge clk_t, posedge reset) begin
120         if (reset) begin
121             oc0a <= 0;
122             oc0b <= 0;
123         end else begin
124             case (timer_mode)
125                 `NORMAL, `CTC: begin
126                     case (com0a)
127                         1: if (mem_tcnt0 == mem_ocr0a) oc0a <= ~oc0a;   // Toggle OC0A on compare match
128                         2: if (mem_tcnt0 == mem_ocr0a) oc0a <= 0;       // Clear OC0A on compare match
129                         3: if (mem_tcnt0 == mem_ocr0a) oc0a <= 1;       // Set OC0A on compare match
130                         default: ;                                      // Do nothing, OC0A disconnected from timer
131                     endcase
132
133                     case (com0b)
134                         1: if (mem_tcnt0 == mem_ocr0b) oc0b <= ~oc0b;   // Toggle OC0B on compare match
135                         2: if (mem_tcnt0 == mem_ocr0b) oc0b <= 0;       // Clear OC0B on compare match
136                         3: if (mem_tcnt0 == mem_ocr0b) oc0b <= 1;       // Set OC0B on compare match
137                         default: ;                                      // Do nothing, OC0B disconnected from timer
138                     endcase
139                 end
140
141                 `FAST_PWM_MAX, `FAST_PWM_OCR: begin
142                     case (com0a)
143                         1: if (wgm0[2] == 1 && mem_tcnt0 == mem_ocr0a) oc0a <= ~oc0a;  // WGM02 == 0: Do nothing, OC0A Disconnected from
144                         2: if (mem_tcnt0 == mem_ocr0a) oc0a <= 0;                     // Clear OC0A on compare match, set OC0A at BOTTOM
145                            else if (mem_tcnt0 == 0) oc0a <= 1;
146                         3: if (mem_tcnt0 == mem_ocr0a) oc0a <= 1;                     // Set OC0A on compare match, clear OC0A at BOTTOM
147                            else if (mem_tcnt0 == 0) oc0a <= 0;
148                         default: ;                                                    // Do nothing, OC0A disconnected from timer
149                     endcase
150
151                     case (com0b)
152                         2: if (mem_tcnt0 == mem_ocr0b) oc0b <= 0; // Clear OC0B on compare match, set OC0B at BOTTOM
153                            else if (mem_tcnt0 == 0) oc0b <= 1;
154                         3: if (mem_tcnt0 == mem_ocr0b) oc0b <= 1; // Set OC0B on compare match, clear OC0B at BOTTOM
155                            else if (mem_tcnt0 == 0) oc0b <= 0;
156                         default: ;                                    // Do nothing, OC0B disconnected from timer
157                     endcase
158                 end
159
160                 default: begin
161                     case (com0a)
162                         default: ; // Do nothing, OC0A disconnected from timer
163                     endcase
164
165                     case (com0b)
166                         default: ; // Do nothing, OC0B disconnected from timer
167                     endcase
168                 end
169             endcase
170         end
171     end
172
173 endmodule
174
```

cpu_test.v

```
`include "defines.vh"
`timescale 1ns / 1ps
module cpu_test;
    // Inputs
    reg clk;
    reg reset;

    // Outputs
    wire [7:0] pa;
    wire [7:0] pb;
    wire       oc0a;
    wire       oc0b;

    // Instantiate the Unit Under Test (UUT)
    cpu #(
        .DATA_WIDTH(8),         // registers are 8 bits in width
        .INSTR_WIDTH(16),       // instructions are 16 bits in width
        .I_ADDR_WIDTH(10),      // 2 * 1024 bytes of flash (or ROM in our case)
        .D_ADDR_WIDTH(7),       // 128 bytes of SRAM
        .R_ADDR_WIDTH(5),       // 32 registers
        .RST_ACTIVE_LEVEL(1)
    ) uut (
        .osc_clk(clk),
        .trace_mode(1'b0),
        .prescaler(5'b0),
        .trace_clk(1'b0),
        .reset(reset),
        .pa(pa),
        .pb(pb),
        .oc0a(oc0a),
        .oc0b(oc0b)
    );

    initial begin
        // Initialize Inputs
        clk = 1;
        reset = 1;
        // Wait 10 ns for global reset to finish
        #10;
        reset = 0;
        #2000;
        $finish;
    end
    always #5 clk = ~clk;

    //DO NOT MODIFY
    always @(negedge clk) begin
        if(uut.control.program_counter == 4 || uut.control.program_counter == 5) begin
            $display("TEST1: PC=%d, STATE=%d, OP_TYPE=%d, OP_GROUP=%d, SIGNALS=%d",
             uut.control.program_counter,
             uut.control.state,
             uut.control.opcode_type,
             uut.control.opcode_group,
             uut.control.signals);
            $display("TEST2: PC=%d, STATE=%d, rd_addr=%d, rd_cs=%d, rd_we=%d, rd_oe=%d, rr_addr=%d, rr_cs=%d, rr_we=%d, rr_oe=%d",
             uut.control.program_counter,
             uut.control.state,
             uut.control.rd_addr,
             uut.control.rd_cs,
             uut.control.rd_we,
             uut.control.rd_oe,
             uut.control.rr_addr,
             uut.control.rr_cs,
             uut.control.rr_we,
             uut.control.rr_oe);
            $display("TEST3: PC=%d, STATE=%d, alu_enable=%d, alu_opsel=%d, alu_flags_in=%d, alu_rr=%d, alu_rd=%d",
             uut.control.program_counter,
             uut.control.state,
             uut.control.alu_enable,
             uut.control.alu_opsel,
             uut.control.alu_flags_in,
             uut.control.alu_rr,
             uut.control.alu_rd);
             $display("TEST4: PC=%d, STATE=%d, alu_flags_out=%d, alu_out=%d",
             uut.control.program_counter,
             uut.control.state,
             uut.control.alu_flags_out,
             uut.control.alu_out);
            $display("TEST5: PC=%d, STATE=%d, rd_data=%d",
             uut.control.program_counter,
             uut.control.state,
             uut.control.rd_data);
        end
    end
endmodule

```

VPL

You are logged in as Radu-Andrei DUMITRESCU (Log out)
03-ACS-L-A3-S1-CN2-CA

English (en)
Deutsch (de)
English (en)

Français (fr)
Română (ro)
Data retention summary
Get the mobile app
Acest site este hostat pe platforma hardware achiziționată din proiectul nr. 154/323 cod SMIS - 4428, "Platforma de e-learning si curricula e-content pentru invatamantul superior tehnic". Pentru mai multe detalii vezi http://www.curs.pub.ro.